

Linux (Ubuntu) Server Administration & SSL/HTTPS with Certbot

A Complete Guide — Beginner to Pro

Covers Ubuntu Server fundamentals, system & network administration, security hardening, web servers, and TLS/HTTPS certificate management with Certbot.

Includes hands-on command examples throughout.

Table of Contents

PART 1 — Linux & Ubuntu Fundamentals

- 1.1 Introduction to Linux & Ubuntu Server
- 1.2 Getting Started & First Login
- 1.3 The Linux File System Hierarchy
- 1.4 Essential Navigation & File Commands
- 1.5 Users, Groups & Permissions
- 1.6 Package Management (APT)
- 1.7 Process & Service Management
- 1.8 Text Editors: nano & vim
- 1.9 Text Processing: grep, sed, awk, find
- 1.10 Disks, Storage & Mounting
- 1.11 Networking Basics
- 1.12 Cron Jobs & Task Scheduling
- 1.13 Logs & journalctl
- 1.14 Shell Scripting Basics

PART 2 — Server Administration (Intermediate)

- 2.1 SSH: Setup, Keys & Hardening
- 2.2 Firewall with UFW
- 2.3 Fail2Ban & Intrusion Prevention
- 2.4 systemd: Managing & Creating Services
- 2.5 Installing Nginx
- 2.6 Installing Apache
- 2.7 Reverse Proxy Configuration
- 2.8 Databases: MySQL & PostgreSQL Basics
- 2.9 Backups & Disaster Recovery
- 2.10 Monitoring & Resource Management
- 2.11 Server Hardening Checklist

PART 3 — SSL/HTTPS with Certbot

- 3.1 SSL/TLS & HTTPS Fundamentals
- 3.2 Let's Encrypt & Certbot Overview
- 3.3 Installing Certbot
- 3.4 Issuing a Certificate for Nginx
- 3.5 Issuing a Certificate for Apache
- 3.6 Standalone & Webroot Modes
- 3.7 Wildcard Certificates (DNS Challenge)
- 3.8 Auto-Renewal
- 3.9 Managing Certificates
- 3.10 Hardening TLS Configuration
- 3.11 Testing & Verifying SSL
- 3.12 Troubleshooting Certbot

PART 4 — Pro-Level Topics

- 4.1 Certbot Hooks & Automation
- 4.2 Multi-Server & Load-Balanced Certificates
- 4.3 Certbot with Docker
- 4.4 Complete Security Checklist
- 4.5 Command Cheat Sheet

PART 1

Linux & Ubuntu Fundamentals

1.1 Introduction to Linux & Ubuntu Server

Linux is an open-source, Unix-like operating system kernel. A **distribution** ("distro") bundles the kernel with system tools, package managers, and software. **Ubuntu** is one of the most popular Debian-based distributions, and **Ubuntu Server** is a variant without a graphical desktop, optimized for running services.

Why Ubuntu Server for hosting?

- Free, open-source, huge community support
- Long Term Support (LTS) releases supported for 5 years (e.g. 22.04 LTS, 24.04 LTS)
- Massive software repository via APT
- Used by most cloud providers (AWS, DigitalOcean, Azure, GCP) as a base image

Note: This guide assumes Ubuntu 22.04/24.04 LTS accessed via SSH on a cloud VPS, but nearly everything applies to Debian-based systems in general.

Checking your Ubuntu version

```
lsb_release -a
cat /etc/os-release
uname -r          # kernel version
```

1.2 Getting Started & First Login

Connecting via SSH

```
ssh username@your_server_ip
ssh root@203.0.113.10
ssh -p 2222 deploy@203.0.113.10 # custom port
```

Updating the system (do this first, always)

```
sudo apt update           # refresh package index
sudo apt upgrade -y       # install available upgrades
sudo apt full-upgrade -y  # handles dependency changes too
sudo reboot               # if kernel was updated
```

Creating a non-root sudo user (critical first step)

```
adduser deploy
usermod -aG sudo deploy
su - deploy
```

Security Warning: Never operate day-to-day as `root`. Always create a dedicated `sudo` user and disable direct `root` SSH login (covered in Section 2.1).

Setting hostname and timezone

```
sudo hostnamectl set-hostname webserver01  
sudo timedatectl set-timezone Asia/Kolkata  
timedatectl # view current date/time settings
```

1.3 The Linux File System Hierarchy

Path	Purpose
/	Root of the entire filesystem
/home	User home directories
/etc	System-wide configuration files
/var	Variable data: logs, mail, caches
/var/log	System and application logs
/var/www	Default web root for hosted sites
/usr	Installed software, libraries, binaries
/bin, /sbin	Essential system binaries
/tmp	Temporary files (cleared on reboot)
/opt	Optional/third-party software
/root	Home directory of the root user
/proc, /sys	Virtual filesystems exposing kernel/process info
/etc/ssl, /etc/letsencrypt	SSL/TLS certificates and keys

1.4 Essential Navigation & File Commands

```
pwd                # print working directory
ls -la             # list all files, long format
cd /var/www        # change directory
mkdir -p project/logs # create nested directories
touch file.txt      # create empty file
cp file.txt backup.txt # copy
cp -r dir1/ dir2/    # copy recursively
mv old.txt new.txt   # move / rename
rm file.txt          # delete file
rm -rf old_folder/   # delete folder recursively (careful!)
cat file.txt         # print file contents
less file.txt        # paginated view
head -n 20 file.txt  # first 20 lines
tail -n 20 file.txt  # last 20 lines
tail -f /var/log/syslog # live-follow a log file
file document.pdf    # identify file type
wc -l file.txt        # count lines
ln -s /path/target /path/link # create symlink
```

Tip: Use **Tab** for autocompletion and **Ctrl+R** to search command history interactively.

1.5 Users, Groups & Permissions

Managing users and groups

```
sudo adduser alice           # create user (interactive)
sudo deluser alice          # remove user
sudo usermod -aG docker alice # add user to a group
groups alice                 # list groups for a user
sudo passwd alice           # change a user's password
id alice                     # show UID/GID info
sudo groupadd developers     # create a new group
```

Understanding permissions

Every file has a permission string like `-rwxr-xr--` : type, owner, group, others. Each triplet = read(4) + write(2) + execute(1).

```
ls -l app.sh
# -rwxr-xr-- 1 deploy developers 850 Jul 8 10:00 app.sh

chmod 755 app.sh      # rwx for owner, rx for group/others
chmod u+x app.sh      # add execute for owner only
chmod -R 644 /var/www/site # recursive read/write for owner, read for others
chown deploy:developers app.sh # change owner:group
chown -R www-data:www-data /var/www/site # common for web files
```

Numeric	Permission
7	rwx (read+write+execute)
6	rw- (read+write)
5	r-x (read+execute)
4	r-- (read only)

sudo & the sudoers file

```
sudo visudo           # safely edit /etc/sudoers
# Grant passwordless sudo for a specific command:
deploy ALL=(ALL) NOPASSWD: /usr/bin/systemctl restart nginx
```

1.6 Package Management (APT)

```
sudo apt update           # refresh package lists
sudo apt install nginx -y # install a package
sudo apt remove nginx     # remove package, keep config
sudo apt purge nginx      # remove package + config
sudo apt autoremove       # remove unused dependencies
apt list --installed | grep nginx # check if installed
apt-cache policy nginx    # show available versions
sudo apt install curl git unzip vim -y # install multiple packages
sudo add-apt-repository ppa:example/ppa # add a PPA
sudo dpkg -i package.deb   # install a local .deb file
```

Tip: Use `apt` for everyday use; `dpkg` is the lower-level tool APT builds on.

1.7 Process & Service Management

Viewing processes

```
ps aux                # snapshot of all running processes
ps aux | grep nginx   # filter for nginx
top                   # live process viewer
htop                  # nicer live viewer (sudo apt install htop)
kill 1234              # send SIGTERM to PID 1234
kill -9 1234           # force-kill (SIGKILL)
pkill -f "node server.js" # kill by matching command
nice -n 10 ./heavy_task.sh # run with lower priority
```

systemd services (the modern init system)

```
sudo systemctl status nginx
sudo systemctl start nginx
sudo systemctl stop nginx
sudo systemctl restart nginx
sudo systemctl reload nginx      # reload config without downtime
sudo systemctl enable nginx      # start automatically on boot
sudo systemctl disable nginx
systemctl list-units --type=service --state=running
```

1.8 Text Editors: nano & vim

nano (beginner-friendly)

```
nano /etc/nginx/nginx.conf
# Ctrl+O = save, Ctrl+X = exit, Ctrl+W = search
```

vim (powerful, steeper learning curve)

```
vim /etc/nginx/nginx.conf
# i      -> enter INSERT mode (start typing)
# Esc    -> return to NORMAL mode
# :w     -> save
# :q     -> quit
# :wq    -> save and quit
# :q!    -> quit without saving
# /searchterm + Enter -> search; n = next match
# dd     -> delete current line
# yy then p -> copy and paste a line
```

1.9 Text Processing: grep, sed, awk, find

```
grep "error" /var/log/nginx/error.log      # find lines containing "error"
grep -ri "failed password" /var/log/auth.log # case-insensitive, recursive
grep -c "GET" access.log                  # count matches

sed 's/foo/bar/g' file.txt                 # replace foo with bar (prints to stdout)
sed -i 's/8080/443/g' config.conf         # edit file in place

awk '{print $1}' access.log                # print first column (e.g., IP addresses)
awk -F, '{print $2}' data.csv              # custom delimiter

find / -name "*.log" -mtime +30            # files older than 30 days
find . -type f -size +100M                 # files larger than 100MB
find /var/www -name "*.php" -exec chmod 644 {} \;
```

1.10 Disks, Storage & Mounting

```
df -h                # disk space usage, human-readable
du -sh /var/log      # size of a directory
du -sh /var/log/* | sort -rh | head -10  # largest items in a dir
lsblk                # list block devices (disks/partitions)
fdisk -l             # detailed partition info (sudo)
mount /dev/sdb1 /mnt/data # mount a device
umount /mnt/data      # unmount
sudo nano /etc/fstab   # persistent mounts across reboots
```

Warning: An incorrect `/etc/fstab` entry can prevent the server from booting. Always test with `mount -a` before rebooting.

1.11 Networking Basics

```
ip a                # show network interfaces & IPs
ip route            # show routing table
ss -tulpn           # list listening ports & owning processes
netstat -tulpn      # older equivalent (net-tools)
ping -c 4 google.com # test connectivity
curl -I https://example.com # fetch HTTP headers only
curl -v https://example.com # verbose, shows TLS handshake
wget https://example.com/file.zip
dig example.com      # DNS lookup
dig example.com +short
nslookup example.com
host example.com
traceroute example.com # trace network path
hostname -I            # show server's IP addresses
```

1.12 Cron Jobs & Task Scheduling

```
crontab -e          # edit current user's crontab
crontab -l          # list current cron jobs
sudo crontab -e -u www-data # edit another user's crontab
```

Cron syntax: `minute hour day-of-month month day-of-week command`

```
# Run a backup script every day at 2:30 AM
30 2 * * * /home/deploy/scripts/backup.sh >> /var/log/backup.log 2>&1

# Run every 15 minutes
*/15 * * * * /usr/local/bin/healthcheck.sh

# Run every Sunday at midnight
0 0 * * 0 /home/deploy/scripts/weekly_cleanup.sh
```

Tip: Use crontab.guru to build and verify cron expressions.

1.13 Logs & journalctl

```
journalctl -u nginx           # logs for a specific systemd unit
journalctl -u nginx -f        # follow live
journalctl --since "1 hour ago"
journalctl -p err -b          # errors since last boot
journalctl --disk-usage
sudo journalctl --vacuum-time=7d # trim logs older than 7 days

tail -f /var/log/syslog
tail -f /var/log/nginx/access.log
tail -f /var/log/nginx/error.log
tail -f /var/log/auth.log      # SSH login attempts, sudo usage
```

1.14 Shell Scripting Basics

```
#!/bin/bash
# backup.sh – simple website backup script
set -e    # exit immediately if a command fails

TIMESTAMP=$(date +%F_%H-%M-%S)
SRC="/var/www/mysite"
DEST="/backups/mysite_${TIMESTAMP}.tar.gz"

tar -czf "$DEST" "$SRC"
echo "Backup created at $DEST"

# Keep only the last 7 backups
ls -lt /backups/mysite_*.tar.gz | tail -n +8 | xargs -r rm --
```

```
chmod +x backup.sh    # make it executable
./backup.sh            # run it
```

Common constructs:

```
# if/else
if [ -f "/etc/nginx/nginx.conf" ]; then
    echo "Nginx config exists"
else
    echo "Missing config"
fi

# for loop
for f in /var/www/*.log; do
    echo "Processing $f"
done

# variables & command substitution
NOW=$(date)
echo "Current time: $NOW"
```

PART 2

Server Administration — Intermediate

2.1 SSH: Setup, Keys & Hardening

Generating and using SSH keys

```
# On your LOCAL machine:
ssh-keygen -t ed25519 -C "your_email@example.com"
# Press Enter to accept default location, optionally set a passphrase

ssh-copy-id deploy@203.0.113.10      # copy public key to server
# or manually:
cat ~/.ssh/id_ed25519.pub | ssh deploy@203.0.113.10 "mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys"

ssh deploy@203.0.113.10              # now logs in without a password
```

Hardening /etc/ssh/sshd_config

```
sudo nano /etc/ssh/sshd_config
```

```
Port 2222                # change from default 22 (reduces automated scans)
PermitRootLogin no       # disable direct root login
PasswordAuthentication no # force key-based auth only
PubkeyAuthentication yes
AllowUsers deploy alice  # whitelist specific users
MaxAuthTries 3
ClientAliveInterval 300
ClientAliveCountMax 2
```

```
sudo sshd -t              # test config syntax before restarting!
sudo systemctl restart ssh
```

Warning: Keep your current SSH session open while testing changes, and open a **second** terminal to confirm you can still log in before closing the first. Locking yourself out of a remote server is a common and painful mistake.

2.2 Firewall with UFW

UFW (Uncomplicated Firewall) is a friendly front-end for iptables/nftables.

```
sudo apt install ufw -y
sudo ufw default deny incoming
sudo ufw default allow outgoing

sudo ufw allow 2222/tcp      # your custom SSH port – allow BEFORE enabling!
sudo ufw allow 80/tcp       # HTTP
sudo ufw allow 443/tcp      # HTTPS
sudo ufw allow "Nginx Full" # pre-defined app profile (80+443)

sudo ufw enable
sudo ufw status verbose
sudo ufw delete allow 80/tcp # remove a rule
sudo ufw allow from 203.0.113.5 to any port 22 # allow SSH only from one IP
```

Warning: Always allow your SSH port **before** running `ufw enable`, or you will be locked out.

2.3 Fail2Ban & Intrusion Prevention

Fail2Ban scans log files and temporarily bans IPs showing malicious behavior (e.g. repeated failed SSH logins).

```
sudo apt install fail2ban -y
sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
sudo nano /etc/fail2ban/jail.local
```

```
[sshd]
enabled = true
port = 2222
maxretry = 4
bantime = 3600
findtime = 600
```

```
sudo systemctl restart fail2ban
sudo fail2ban-client status
sudo fail2ban-client status sshd
sudo fail2ban-client set sshd unbanip 203.0.113.99
```

2.4 systemd: Managing & Creating Services

Run your own application (e.g. a Node.js or Python app) as a persistent, auto-restarting background service.

```
sudo nano /etc/systemd/system/myapp.service
```

```
[Unit]
Description=My Node.js Application
After=network.target

[Service]
Type=simple
User=deploy
WorkingDirectory=/home/deploy/myapp
ExecStart=/usr/bin/node /home/deploy/myapp/server.js
Restart=on-failure
RestartSec=5
Environment=NODE_ENV=production
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable myapp
sudo systemctl start myapp
sudo systemctl status myapp
journalctl -u myapp -f
```

2.5 Installing Nginx

```
sudo apt update
sudo apt install nginx -y
sudo systemctl enable nginx
sudo systemctl status nginx
```

Basic server block (virtual host)

```
sudo nano /etc/nginx/sites-available/example.com
```

```
server {
    listen 80;
    server_name example.com www.example.com;
    root /var/www/example.com/html;
    index index.html index.htm;

    location / {
        try_files $uri $uri/ =404;
    }

    access_log /var/log/nginx/example.com.access.log;
    error_log /var/log/nginx/example.com.error.log;
}
```

```
sudo mkdir -p /var/www/example.com/html
echo "<h1>It works!</h1>" | sudo tee /var/www/example.com/html/index.html

sudo ln -s /etc/nginx/sites-available/example.com /etc/nginx/sites-enabled/
sudo nginx -t                # ALWAYS test config before reloading
sudo systemctl reload nginx
```


2.6 Installing Apache

```
sudo apt update
sudo apt install apache2 -y
sudo systemctl enable apache2
```

```
sudo nano /etc/apache2/sites-available/example.com.conf
```

```
<VirtualHost *:80>
    ServerName example.com
    ServerAlias www.example.com
    DocumentRoot /var/www/example.com/html
    ErrorLog ${APACHE_LOG_DIR}/example.com-error.log
    CustomLog ${APACHE_LOG_DIR}/example.com-access.log combined
</VirtualHost>
```

```
sudo a2ensite example.com.conf
sudo a2dissite 000-default.conf
sudo apache2ctl configtest
sudo systemctl reload apache2
```

2.7 Reverse Proxy Configuration

A reverse proxy lets Nginx sit in front of an application server (e.g. Node.js on port 3000) — essential before adding SSL.

```
server {  
    listen 80;  
    server_name app.example.com;  
  
    location / {  
        proxy_pass http://127.0.0.1:3000;  
        proxy_http_version 1.1;  
        proxy_set_header Upgrade $http_upgrade;  
        proxy_set_header Connection 'upgrade';  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
        proxy_cache_bypass $http_upgrade;  
    }  
}
```

2.8 Databases: MySQL & PostgreSQL Basics

MySQL / MariaDB

```
sudo apt install mysql-server -y
sudo mysql_secure_installation
sudo mysql -u root -p
```

```
CREATE DATABASE myapp;
CREATE USER 'appuser'@'localhost' IDENTIFIED BY 'StrongPassword123!';
GRANT ALL PRIVILEGES ON myapp.* TO 'appuser'@'localhost';
FLUSH PRIVILEGES;
```

PostgreSQL

```
sudo apt install postgresql postgresql-contrib -y
sudo -u postgres psql
```

```
CREATE DATABASE myapp;
CREATE USER appuser WITH PASSWORD 'StrongPassword123!';
GRANT ALL PRIVILEGES ON DATABASE myapp TO appuser;
```

2.9 Backups & Disaster Recovery

```
# File backup with rsync (efficient, incremental)
rsync -avz --delete /var/www/ deploy@backupserver:/backups/www/

# Compressed archive backup
tar -czvf site_backup_$(date +%F).tar.gz /var/www/example.com

# Database backup
mysqldump -u root -p myapp > myapp_$(date +%F).sql
pg_dump -U appuser myapp > myapp_$(date +%F).sql

# Restore
mysql -u root -p myapp < myapp_2026-07-08.sql
tar -xzf site_backup_2026-07-08.tar.gz -C /
```

Tip: Follow the 3-2-1 rule: 3 copies of data, on 2 different media, with 1 copy off-site. Automate backups via cron (Section 1.12) and periodically test restoring them.

2.10 Monitoring & Resource Management

```
htop                # interactive process/resource monitor
free -h            # memory usage
uptime              # load average & uptime
vmstat 2 5          # system performance snapshots
iostat -x 2         # disk I/O stats (sysstat package)
nload / iftop       # live network bandwidth usage
sudo apt install netdata -y # optional real-time web dashboard
```

2.11 Server Hardening Checklist

- Create a non-root sudo user; disable root SSH login
- Use SSH key authentication; disable password authentication
- Change the default SSH port (optional, reduces noise)
- Enable UFW with a default-deny policy
- Install Fail2Ban for brute-force protection
- Keep the system updated: `sudo apt update && sudo apt upgrade -y`
- Enable automatic security updates: `sudo apt install unattended-upgrades -y`
- Disable unused services: `systemctl list-unit-files --state=enabled`
- Set up log monitoring and alerting
- Enforce HTTPS everywhere (see Part 3)
- Take regular, tested backups

PART 3

SSL / HTTPS with Certbot

3.1 SSL/TLS & HTTPS Fundamentals

SSL (Secure Sockets Layer) and its modern successor **TLS** (Transport Layer Security) are cryptographic protocols that encrypt data between a client and a server. **HTTPS** is simply HTTP running over a TLS-encrypted connection.

What TLS provides

- **Encryption** — data can't be read by eavesdroppers
- **Integrity** — data can't be tampered with in transit
- **Authentication** — the certificate proves the server is who it claims to be

Simplified TLS handshake

1. Client connects and sends supported TLS versions/ciphers ("Client Hello")
2. Server responds with its certificate and chosen cipher ("Server Hello")
3. Client verifies the certificate against trusted Certificate Authorities (CAs)
4. Client and server negotiate a shared symmetric session key
5. Encrypted application data (HTTP) now flows over the connection

Certificate types

Type	Validation Level	Typical Use
DV (Domain Validated)	Proves control of the domain only	Blogs, most websites, APIs — what Let's Encrypt issues
OV (Organization Validated)	Verifies the organization's identity	Business websites
EV (Extended Validation)	Rigorous legal/business verification	Banks, high-trust e-commerce (less common today)

3.2 Let's Encrypt & Certbot Overview

Let's Encrypt is a free, automated, open Certificate Authority run by the Internet Security Research Group (ISRG). It issues short-lived (90-day) domain-validated certificates via the **ACME** protocol.

Certbot is the official EFF-maintained client that automates obtaining and renewing Let's Encrypt certificates, and can automatically configure Nginx or Apache.

Why 90 days? Short lifetimes limit damage from a compromised or misissued certificate and encourage automation — which Certbot provides via auto-renewal (Section 3.8).

3.3 Installing Certbot

The officially recommended installation method is via **snap**, which ensures you always run the latest version.

```
# Remove any old apt-installed certbot first (if present)
sudo apt remove certbot -y

# Install snapd if not already present
sudo apt install snapd -y
sudo snap install core; sudo snap refresh core

# Install Certbot
sudo snap install --classic certbot

# Create a command-line shortcut
sudo ln -s /snap/bin/certbot /usr/bin/certbot

certbot --version
```

Alternative: apt installation (simpler, sometimes older version)

```
sudo apt install certbot python3-certbot-nginx -y
# or for Apache:
sudo apt install certbot python3-certbot-apache -y
```

Note: Install the plugin matching your web server — `python3-certbot-nginx` for Nginx, `python3-certbot-apache` for Apache. Snap installs handle this automatically when you run `certbot`.

3.4 Issuing a Certificate for Nginx

Prerequisites: your domain's DNS **A record** must already point to your server's public IP, and port 80/443 must be open in your firewall.

```
sudo ufw allow "Nginx Full"      # ensure 80 & 443 are open

sudo certbot --nginx -d example.com -d www.example.com
```

Certbot will:

1. Verify domain ownership via an HTTP-01 challenge (temporary file served on port 80)
2. Obtain the certificate from Let's Encrypt
3. Automatically edit your Nginx server block to add the `listen 443 ssl;` directive and certificate paths
4. Optionally set up an HTTP → HTTPS redirect (choose "Yes" when prompted)

Resulting Nginx config (auto-generated):

```
server {
    server_name example.com www.example.com;
    root /var/www/example.com/html;

    listen 443 ssl;
    ssl_certificate /etc/letsencrypt/live/example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/example.com/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;
}

server {
    if ($host = www.example.com) { return 301 https://$host$request_uri; }
    if ($host = example.com)     { return 301 https://$host$request_uri; }
    listen 80;
    server_name example.com www.example.com;
    return 404;
}
```

```
sudo nginx -t
sudo systemctl reload nginx
curl -I https://example.com    # verify it now serves HTTPS
```

3.5 Issuing a Certificate for Apache

```
sudo apt install python3-certbot-apache -y
sudo certbot --apache -d example.com -d www.example.com
```

Certbot's Apache plugin edits your VirtualHost to add a matching `<VirtualHost *:443>` block with `SSLEngine on` and the certificate paths, mirroring the Nginx flow above.

```
sudo apache2ctl configtest
sudo systemctl reload apache2
```

3.6 Standalone & Webroot Modes

Use these when there is no running web server yet, or when you want to issue a certificate without touching existing server config.

Standalone mode

Certbot temporarily runs its own mini web server on port 80. Stop Nginx/Apache first.

```
sudo systemctl stop nginx
sudo certbot certonly --standalone -d example.com -d www.example.com
sudo systemctl start nginx
```

Webroot mode

Certbot drops a validation file into your existing site's document root while the web server keeps running — no downtime.

```
sudo certbot certonly --webroot -w /var/www/example.com/html -d example.com -d www.example.com
```

Tip: Use `certonly` when you want Certbot to just obtain the certificate files without auto-editing your web server config — useful for custom setups or non-supported servers (mail servers, custom reverse proxies, etc).

3.7 Wildcard Certificates (DNS Challenge)

A wildcard certificate (e.g. `*.example.com`) covers all subdomains. Let's Encrypt requires proving domain control via a **DNS-01** challenge (a TXT record), since HTTP validation can't cover arbitrary subdomains.

```
# Manual DNS challenge (works with any DNS provider)
sudo certbot certonly --manual --preferred-challenges dns \
  -d example.com -d "*.example.com"
```

Certbot will pause and ask you to create a TXT record like:

```
_acme-challenge.example.com.  TXT  "some-random-token-string"
```

Add that record in your DNS provider's dashboard, wait for propagation, then press Enter to continue.

Automated DNS challenge (recommended for production)

If your DNS provider is supported (Cloudflare, Route53, DigitalOcean, GoDaddy, etc.), install its Certbot DNS plugin for fully automated wildcard renewal:

```
sudo snap install certbot-dns-cloudflare
sudo nano /etc/letsencrypt/cloudflare.ini
```

```
dns_cloudflare_api_token = your_cloudflare_api_token
```

```
sudo chmod 600 /etc/letsencrypt/cloudflare.ini

sudo certbot certonly --dns-cloudflare \
  --dns-cloudflare-credentials /etc/letsencrypt/cloudflare.ini \
  -d example.com -d "*.example.com"
```

3.8 Auto-Renewal

Let's Encrypt certificates expire every **90 days**. Snap-installed Certbot automatically sets up a systemd timer to check twice daily and renew certificates within 30 days of expiry.

```
systemctl list-timers | grep certbot    # confirm the renewal timer is active
sudo certbot renew --dry-run            # simulate renewal without making changes
sudo certbot renew                      # force a real renewal check now
```

If using apt-installed Certbot (no automatic timer)

```
sudo crontab -e
```

```
0 3 * * * /usr/bin/certbot renew --quiet --post-hook "systemctl reload nginx"
```

Tip: Always test with `--dry-run` after any renewal-related config change. Certbot automatically reloads Nginx/Apache after a successful renewal when installed via the respective plugin.

3.9 Managing Certificates

```
sudo certbot certificates          # list all certs, expiry dates, paths

sudo certbot delete --cert-name example.com

sudo certbot revoke --cert-path /etc/letsencrypt/live/example.com/cert.pem

sudo certbot certonly --nginx -d newsub.example.com    # add a new cert

# Expand an existing cert to cover more domains:
sudo certbot --nginx -d example.com -d www.example.com -d shop.example.com --expand
```

Certificate files live under `/etc/letsencrypt/live/<domain>/` :

File	Purpose
fullchain.pem	Your certificate + intermediate CA chain (used in server config)
privkey.pem	Your private key — keep this secret, never share it
cert.pem	Your certificate only, without the chain
chain.pem	Just the intermediate CA chain

3.10 Hardening TLS Configuration

Certbot's default `options-ssl-nginx.conf` is reasonably secure, but for a stronger, modern configuration:

```
ssl_protocols TLSv1.2 TLSv1.3;
ssl_prefer_server_ciphers off;
ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384;
ssl_session_cache shared:SSL:10m;
ssl_session_timeout 1d;
ssl_session_tickets off;

# HSTS: force browsers to always use HTTPS for this domain
add_header Strict-Transport-Security "max-age=63072000; includeSubDomains; preload" always;

# OCSP stapling – faster certificate revocation checks
ssl_stapling on;
ssl_stapling_verify on;
resolver 8.8.8.8 8.8.4.4 valid=300s;
```

Tip: Use the [Mozilla SSL Configuration Generator](#) to produce an up-to-date recommended config for your exact Nginx/Apache/OpenSSL version.

Warning: Only enable HSTS `preload` once you're certain your entire site (and all subdomains, if `includeSubDomains` is set) will permanently serve HTTPS — it is very difficult to reverse once browsers cache it.

3.11 Testing & Verifying SSL

```
curl -vI https://example.com 2>&1 | grep -i "SSL\|subject\|issuer"

openssl s_client -connect example.com:443 -servername example.com < /dev/null

# Check expiry date specifically
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null \
    | openssl x509 -noout -dates

# Verify certificate chain
openssl verify -CAfile chain.pem cert.pem
```

For a full external audit, use **SSL Labs**: <https://www.ssllabs.com/ssltest/> — enter your domain and review the grade (A/A+ is the target), protocol support, cipher strength, and certificate chain.

3.12 Troubleshooting Certbot

Problem	Likely Cause / Fix
"Timeout during connect" / challenge failed	Port 80 blocked by firewall, or DNS A record not pointing to this server yet. Check <code>ufw status</code> and <code>dig example.com</code> .
"Too many certificates already issued"	Let's Encrypt rate limit (50 certs/domain/week). Use <code>--staging</code> flag while testing to avoid hitting production limits.
Nginx fails to reload after renewal	Run <code>sudo nginx -t</code> to find the syntax error; check <code>/var/log/letsencrypt/letsencrypt.log</code> .
"Certificate not yet due for renewal"	Normal — Certbot only renews within 30 days of expiry. Use <code>--force-renewal</code> to override for testing.
Mixed content warnings in browser	Page loads HTTP resources (images, scripts) on an HTTPS page. Update links to use <code>https://</code> or protocol-relative URLs.
Wildcard cert challenge fails	DNS TXT record not propagated yet, or added under the wrong subdomain. Verify with <code>dig TXT _acme-challenge.example.com</code> .

```
# Useful staging flag for testing without hitting rate limits
sudo certbot --nginx -d example.com --staging

# View detailed logs
sudo tail -100 /var/log/letsencrypt/letsencrypt.log
```

PART 4

Pro-Level Topics

4.1 Certbot Hooks & Automation

Hooks let you run custom commands before/after certificate operations — useful for reloading services that don't auto-detect new certs (e.g. Postfix, Dovecot, HAProxy, custom Docker containers).

```
# Run once, right before the renewal attempt
sudo certbot renew --pre-hook "systemctl stop nginx"

# Run once, after a successful renewal
sudo certbot renew --post-hook "systemctl start nginx"

# Run after EVERY certificate that's renewed (deploy-hook)
sudo certbot renew --deploy-hook "systemctl reload haproxy"
```

For persistent hooks that always run automatically, drop scripts into:

```
/etc/letsencrypt/renewal-hooks/pre/
/etc/letsencrypt/renewal-hooks/deploy/
/etc/letsencrypt/renewal-hooks/post/
```

```
sudo nano /etc/letsencrypt/renewal-hooks/deploy/reload-services.sh
```

```
#!/bin/bash
systemctl reload nginx
systemctl restart haproxy
echo "$(date): Certificates deployed" >> /var/log/cert-deploy.log
```

```
sudo chmod +x /etc/letsencrypt/renewal-hooks/deploy/reload-services.sh
```

4.2 Multi-Server & Load-Balanced Certificates

When multiple backend servers sit behind a load balancer, you generally terminate SSL in one of two places:

- **At the load balancer** (recommended): Certbot runs on the LB only; backend traffic can be plain HTTP over a private network.
- **At each backend server**: requires syncing the same certificate + private key to every node.

Syncing certificates across nodes

```
# From the primary Certbot host, in a deploy-hook script:  
rsync -avz /etc/letsencrypt/live/example.com/ \  
    deploy@backend2:/etc/letsencrypt/live/example.com/  
ssh deploy@backend2 "sudo systemctl reload nginx"
```

Note: Only run Certbot's ACME challenge/renewal process on ONE authoritative node to avoid rate-limit conflicts; distribute the resulting certificate files to the others.

4.3 Certbot with Docker

A common pattern: run Certbot in its own container, sharing a volume with the Nginx container.

```
# docker-compose.yml
version: "3.8"
services:
  nginx:
    image: nginx:latest
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf
      - certbot-etc:/etc/letsencrypt
      - certbot-www:/var/www/certbot

  certbot:
    image: certbot/certbot
    volumes:
      - certbot-etc:/etc/letsencrypt
      - certbot-www:/var/www/certbot
    entrypoint: >
      sh -c "trap exit TERM; while ;; do certbot renew;
        sleep 12h & wait $$!}; done;"

volumes:
  certbot-etc:
  certbot-www:
```

```
# Initial certificate issuance:
docker compose run --rm certbot certonly --webroot \
  -w /var/www/certbot -d example.com -d www.example.com

docker compose exec nginx nginx -s reload
```

4.4 Complete Security Checklist

- ✓ Non-root sudo user; root SSH login disabled
- ✓ SSH key auth only; password auth disabled
- ✓ UFW enabled, default-deny incoming
- ✓ Fail2Ban active on SSH (and any other exposed auth)
- ✓ Automatic security updates enabled
- ✓ HTTPS enforced site-wide via HTTP→HTTPS redirect
- ✓ Modern TLS config: TLS 1.2/1.3 only, strong ciphers
- ✓ HSTS header enabled
- ✓ Certificate auto-renewal verified (`certbot renew --dry-run`)
- ✓ Regular, tested off-site backups
- ✓ Least-privilege database users, no root DB access from apps
- ✓ Monitoring/alerting for downtime and certificate expiry
- ✓ Unused ports/services closed and disabled

4.5 Command Cheat Sheet

Task	Command
Update system	<code>sudo apt update && sudo apt upgrade -y</code>
Add sudo user	<code>adduser NAME && usermod -aG sudo NAME</code>
Check open ports	<code>ss -tulpn</code>
Enable firewall	<code>ufw allow 22 && ufw enable</code>
Restart service	<code>systemctl restart SERVICE</code>
Live logs	<code>journalctl -u SERVICE -f</code>
Test Nginx config	<code>nginx -t</code>
Install Certbot	<code>snap install --classic certbot</code>
Get certificate (Nginx)	<code>certbot --nginx -d DOMAIN</code>
Get certificate (webroot)	<code>certbot certonly --webroot -w PATH -d DOMAIN</code>
Test renewal	<code>certbot renew --dry-run</code>
List certificates	<code>certbot certificates</code>
Check cert expiry (openssl)	<code>openssl s_client -connect DOMAIN:443 openssl x509 -noout -dates</code>
Revoke & delete cert	<code>certbot revoke --cert-path PATH && certbot delete --cert-name DOMAIN</code>